

PROYECTO INTEGRADOR · COMPUTACIÓN EN LA NUBE

Aplicación Web en la Nube

Guía práctica de diseño, despliegue, automatización y operación cloud

Asignatura	Nivel	Modalidad	Ponderación
Computación en la Nube (INFO-02028)	8.º Semestre	Proyecto Grupal Incremental	30 % de la nota final



IDEA CENTRAL DEL PROYECTO

El objetivo principal trasciende el desarrollo de una aplicación web estándar. Se busca demostrar la capacidad técnica del equipo para convertirla en un sistema en la nube nativo, sólido e integral: diseñando su arquitectura, automatizando el despliegue, gestionando datos con eficacia, asegurando la observabilidad continua y proyectando su escalabilidad en producción.

Documento para discusión y seguimiento con los estudiantes · Semestre septiembre 2026 – enero 2027

1. Propósito y alcance

Cada equipo deberá diseñar, desarrollar, desplegar y defender una aplicación web funcional alojada en infraestructura o servicios de nube. El dominio funcional es de libre elección: comercio electrónico, red social, reservas, pagos simulados, gestión académica, inventario, colaboración, contenidos, IoT u otro problema aprobado por el profesor.

La complejidad funcional no será el criterio principal. La evaluación se concentrará en las decisiones de arquitectura, despliegue, automatización, datos, seguridad, operación y escalabilidad, en correspondencia con las unidades del programa de la asignatura.

PRINCIPIO DE EVALUACIÓN CLAVE

Una arquitectura sencilla, bien documentada, coherente y plenamente dominada por el equipo obtendrá una mejor ponderación que una solución sobredimensionada cuya complejidad no se pueda justificar ni defender técnicamente.

2. Resultados de aprendizaje que debe evidenciar

- Diseñar e implementar una solución de software sobre una arquitectura centralizada o distribuida.
- Aplicar conceptos de sistemas distribuidos, computación en la nube, contenedores, modelos de servicio y modelos de despliegue.
- Desplegar una aplicación fuera del entorno local y mantenerla accesible para su revisión.
- Aplicar automatización, configuración, monitoreo y prácticas de seguridad apropiadas al contexto.
- Analizar escalabilidad, disponibilidad, costos y compromisos técnicos de la solución.
- Trabajar en equipo y demostrar dominio individual de las decisiones y componentes implementados.

3. Condiciones generales

- El proyecto se desarrollará en equipos. Los equipos serán de 5 personas, y los integrantes deben pertenecer al mismo grupo de laboratorio ya que las presentaciones se harán en este espacio.
- La aplicación deberá estar desplegada en Internet. Una solución que funcione únicamente en localhost no satisface el objetivo del proyecto.
- El lenguaje de programación, framework y dominio funcional son de libre elección, siempre que permitan cumplir los requisitos técnicos y sean justificables.
- El equipo deberá mantener un repositorio GitHub desde las primeras etapas del proyecto.
- Las decisiones tecnológicas deberán justificarse. Utilizar una tecnología más compleja no implica una mejor calificación.
- El proyecto será revisado incrementalmente; los cortes no son entregas aisladas, sino evidencia de evolución del mismo producto.

4. Requisitos técnicos obligatorios

Área Técnica	Exigencia del Proyecto
Repositorio y Trazabilidad	Código en GitHub con historial activo de commits, README explicativo, .gitignore y .env.example. Evidencia clara del aporte de todos los integrantes.
Aplicación Desplegada	URL pública completamente funcional durante las revisiones. No se aceptarán despliegues dependientes del entorno local (localhost).
Arquitectura de Software	Diagrama actualizado delimitando capas: Frontend, Backend/API, Persistencia, Caché y Servicios Externos.
Persistencia de Datos	Base de datos administrada en la nube y adaptada formalmente al caso de uso elegido.
Contenedores	Contenerización con Docker en el backend u otros componentes clave. Explicación de imágenes, puertos y variables.
Pipeline CI/CD	Automatización de integración o despliegue continuo. Se promueve el uso directo de GitHub Actions.
Gestión de Secretos	Variables de entorno aisladas del código fuente. Prohibición estricta de incluir credenciales o llaves en el repositorio.
Observabilidad	Gestión visible de logs, monitoreo de errores y trazabilidad del estado del servicio en tiempo real.
Escalabilidad	Análisis técnico de cuellos de botella y propuesta de crecimiento ante demanda masiva.
IaC (Opcional)	Infraestructura como Código (Terraform, OpenTofu, CloudFormation) otorga bonificación adicional.

5. Arquitectura de referencia

No existe una arquitectura única obligatoria. Como referencia, una solución puede separar una interfaz web, un backend/API, una base de datos y un servicio de caché (en caso que aplique). También son válidas arquitecturas serverless, monolíticas, basadas en Backend-as-a-Service o con múltiples servicios, siempre que la complejidad sea coherente con el problema.

? PREGUNTA CLAVE DE ARQUITECTURA

«¿Por qué esta solución tecnológica y estructural es la más adecuada para responder al problema planteado y a la volumen/escala esperada?»

6. Tecnologías y plataformas

Los estudiantes pueden seleccionar las tecnologías que consideren apropiadas. Para minimizar costos, se recomienda priorizar plataformas con planes gratuitos, créditos educativos o capas gratuitas vigentes al momento del despliegue. Las condiciones comerciales cambian; cada equipo es responsable de revisar límites y requisitos antes de registrarse.

Necesidad	Opciones de referencia
Frontend / hosting web	Vercel, Netlify u opciones equivalentes.
Backend / PaaS	Render u opciones equivalentes.
Base de datos administrada	Supabase u otros servicios PostgreSQL/MySQL/MongoDB administrados.
Caché (Opcional)	Upstash Redis u otro servicio administrado.
Proveedor cloud general	AWS, Microsoft Azure, Google Cloud u otros, si el equipo dispone de acceso y puede controlar costos.
CI/CD	GitHub Actions y/o automatización integrada de la plataforma de despliegue.

No se exigirá el uso de ningún proveedor específico como condición para aprobar el proyecto.

7. Cronograma de revisiones y entregas

El proyecto representa el 30 % de la calificación de la asignatura y se divide en tres hitos. De acuerdo con la pestaña «Plan de Clases» vigente, las fechas de trabajo del proyecto son las siguientes:

Hito de Evaluación	Fecha Prevista	Peso	Objetivo General
1.er Corte de Revisión	28–30 oct 2026	5 %	Validación del problema, arquitectura inicial, repositorio GitHub, MVP funcional y bases de contenerización.
2.º Corte de Revisión	09–11 dic 2026	5 %	Evaluación del sistema: despliegue cloud, persistencia, caché, CI/CD, observabilidad y seguridad.
Entrega y Defensa Final	15–16 dic 2026	20 %	Presentación del informe técnico final, demostración en vivo y defensa individual/grupal del sistema completo.

Nota de planificación: las fechas anteriores se toman de la cronología detallada del Plan de Clases actualizado. Cualquier ajuste formal será comunicado por el profesor.

8. Primer corte de revisión

Para este corte se espera una solución temprana pero verificable. El equipo deberá presentar:

- Problema, usuarios objetivo y alcance del MVP.
- Repositorio GitHub activo, con README inicial y commits distribuidos durante el desarrollo.
- Aplicación básica funcional; idealmente flujo frontend → backend/API → persistencia.
- Diagrama de arquitectura inicial y explicación de los componentes.
- Primer despliegue accesible desde Internet.
- Contenerización de al menos un componente cuando aplique y explicación de imagen, contenedor, puertos, variables de entorno y ejecución.
- Identificación y justificación de los modelos de servicio cloud utilizados o previstos.

9. Segundo corte de revisión

Para el segundo corte se espera que la solución esté cercana a su estado final. El equipo deberá demostrar:

- Aplicación funcional desplegada en la nube.
- Persistencia de producción operativa.
- Caché implementada o justificación técnica documentada de su no aplicación.
- Flujo CI/CD o automatización equivalente funcionando.
- Gestión correcta de variables de entorno y secretos.
- Logs y evidencia básica de operación/monitoreo.
- Medidas mínimas de seguridad.
- Arquitectura actualizada y análisis de escalabilidad.
- Participación verificable de los integrantes.
-

DINÁMICA DE LAS REVISIONES TÉCNICAS

Durante las evaluaciones, el profesor podrá requerir ajustes en tiempo real, auditar pipelines de CI/CD, solicitar logs de producción, evaluar configuraciones de seguridad o simular fallos hipotéticos para comprobar el dominio real del equipo sobre su solución.

10. Entrega final

La entrega final consta de cuatro evidencias obligatorias:

1. URL pública del sistema en funcionamiento.
2. Repositorio GitHub completo y actualizado.
3. Informe técnico final en PDF.
4. Presentación y demostración presencial del proyecto.

11. Informe técnico

El informe deberá ser técnico, concreto y suficientemente detallado para que un tercero comprenda las decisiones del proyecto. Debe incluir, como mínimo:

Sección	Contenido esperado
1. Resumen ejecutivo	Problema, usuarios y solución desarrollada.
2. Alcance funcional	Funciones incluidas, exclusiones y principales casos de uso.
3. Arquitectura	Diagrama, componentes, comunicaciones y responsabilidades.
4. Stack tecnológico	Lenguajes, frameworks, librerías y justificación de selección; alternativas consideradas cuando sea relevante.
5. Modelos cloud	Identificación de IaaS, PaaS, SaaS, FaaS, DBaaS u otros modelos presentes en la solución.
6. Contenedores (si aplica)	Dockerfile(s), estrategia de build/ejecución y decisiones tomadas.
7. Datos	Base de datos, modelo de datos, persistencia y respaldo/recuperación cuando aplique.
8. Caché (si aplica)	Datos cacheados, TTL, invalidación, beneficios y limitaciones.
9. CI/CD	Pipeline, disparadores, etapas, pruebas y mecanismo de despliegue.
10. Configuración y seguridad	Variables de entorno, secretos, HTTPS, autenticación/autorización y controles relevantes.
11. Observabilidad	Logs, errores, estado de servicios y métricas disponibles.
12. Escalabilidad y disponibilidad	Cuellos de botella, componentes stateful/stateless y estrategia ante aumento de carga.
13. Infraestructura como código (si aplica)	Descripción y archivos, si aplica.
14. Costos	Servicios utilizados, capa gratuita/créditos y estimación razonada de costos si la aplicación creciera.
15. Problemas y decisiones	Incidentes, dificultades técnicas, alternativas descartadas y aprendizajes.
16. Conclusiones	Qué aprendió el equipo sobre diseño, despliegue y operación en la nube.

12. Presentación y defensa

La defensa tendrá una duración máxima de 20 minutos por grupo. Se recomienda la siguiente distribución:

Tiempo	Contenido
0–5 min	Problema, alcance y propuesta de valor.
2–5 min	Arquitectura y servicios cloud utilizados.
5–10 min	Demostración en vivo de la aplicación desplegada.
10–15 min	Repositorio, CI/CD, operación y evidencia técnica.
15–20 min	Preguntas del profesor y defensa técnica.

La demostración debe realizarse en vivo sobre la versión desplegada. Un video grabado solo podrá utilizarse como contingencia cuando haya sido autorizado o cuando exista una incidencia externa verificable.

13. Evaluación grupal e individual

El proyecto es grupal, pero la evaluación podrá diferenciar el desempeño individual. La contribución se verificará mediante historial del repositorio, participación en las revisiones y respuestas durante la defensa.



PONDERACIÓN DE EVALUACIÓN INDIVIDUAL

La calificación de cada entrega se dividirá en: **60% desempeño del producto/equipo** y **40% dominio individual**. El profesor podrá hacer ajustes individuales si detecta una distribución desigual del trabajo o del conocimiento técnico.

Ejemplos de preguntas individuales:

- ¿Qué componente de la arquitectura es stateful y por qué?
- ¿Qué ocurre si la base de datos o el servicio de caché deja de responder?
- ¿Qué desencadena el despliegue y qué etapas ejecuta el pipeline?
- ¿Dónde se almacenan las credenciales y cómo se evita versionarlas?
- ¿Por qué se seleccionó esta base de datos o plataforma y qué alternativa se descartó?
- ¿Qué cambiaría si la aplicación recibiera cien veces más tráfico?

14. Uso de herramientas de inteligencia artificial

Se permite utilizar asistentes de IA como herramientas de apoyo para programación, documentación, investigación y resolución de problemas. El equipo es responsable de comprender, validar y defender todo código, configuración, texto o decisión incorporada al proyecto.

USO RESPONSABLE DE INTELIGENCIA ARTIFICIAL

Afirmar que «la IA generó el código o la configuración» no se considera una justificación técnica válida. La falta de comprensión de cualquier componente por parte del equipo penalizará la evaluación individual, independientemente de que el software funcione.

15. Prácticas no aceptables

- Presentar como producción una solución que solo funciona en localhost.
- Versionar contraseñas, tokens, claves privadas u otros secretos reales.
- Crear el repositorio al final del semestre y subir el proyecto completo en uno o pocos commits sin trazabilidad razonable.
- Incluir tecnologías o servicios que el equipo no pueda explicar.
- Presentar código, documentación o infraestructura de terceros como trabajo propio sin reconocer su origen.
- Preparar únicamente una demo visual sin poder demostrar el despliegue, repositorio y componentes técnicos.

16. Qué NO otorga puntos por sí solo

No existe bonificación automática por:

- usar Kubernetes;
- usar microservicios;
- usar AWS, Azure o Google Cloud;
- incorporar muchas tecnologías;
- escribir una gran cantidad de código;
- utilizar herramientas de IA.

Se valorará la pertinencia de las decisiones, la calidad de la implementación, la evidencia de funcionamiento y la capacidad de defender técnicamente la solución.

17. Checklist de entrega final

- ☐ URL pública disponible y probada.
- ☐ Repositorio GitHub accesible al profesor.
- ☐ README actualizado.
- ☐ Historial de commits y contribuciones verificable.
- ☐ Diagrama de arquitectura actualizado.
- ☐ Base de datos de producción operativa.
- ☐ Caché implementada o justificada (si aplica)

- ☐ CI/CD operativo.
- ☐ Secretos fuera del repositorio.
- ☐ Logs/evidencia de operación disponibles.
- ☐ Informe técnico final en PDF.
- ☐ Presentación preparada para máximo 15 minutos.
- ☐ Todos los integrantes preparados para responder preguntas técnicas.

18. Trazabilidad con el programa de la asignatura

Unidad temática	Evidencia principal en el proyecto
I. Introducción a los Sistemas Distribuidos	Arquitectura, comunicación entre componentes, disponibilidad, consistencia y decisiones de distribución.
II. Fundamentos de Computación en la Nube	Selección y explicación de servicios y arquitectura cloud.
III. Virtualización y Contenedores	Docker y comprensión de contenerización; orquestación cuando aplique.
IV. Modelos de Servicios	Identificación y uso razonado de IaaS, PaaS, SaaS, FaaS y servicios de datos.
V. Modelos de Despliegue	Justificación de nube pública, privada, híbrida o multicloud cuando corresponda.
VI. Gestión y Operaciones	Escalabilidad, automatización, configuración, monitoreo, seguridad y proveedores.
VII. Datos en la Nube	Persistencia y servicios de datos; otros componentes de datos cuando el proyecto los requiera.
VIII. Desarrollo de Aplicaciones en la Nube	Patrones, ciclo de desarrollo, arquitectura web escalable y DevOps.

19. Fuentes académicas y normativas

Este proyecto se formula con base en el Programa de Asignatura «Computación en la Nube» (INFO-02028), el Plan de Clases vigente del Prof. Jorge Rojas, el Reglamento de Evaluación de la Facultad de Ingeniería y el Calendario Académico 2026–2027 de la Facultad de Ingeniería.

La evaluación se entiende como un proceso continuo y acumulativo orientado al desarrollo de competencias. Las fechas, ponderaciones y cualquier ajuste formal se regirán por el Plan de Clases y la normativa institucional vigente.